

Programación II - 2026

Trabajo Práctico Integrador: Food Store – Sistema de Gestión de Pedidos de Comida (Consola + JDBC)

Comisión: 1Prog2 Turno Tarde

Integrantes: Morillas Andrés, Quirós Pablo

Fecha entrega: 22/06/2026

ÍNDICE

1. Índice.
2. Marco teórico, Decisiones Técnicas y Arquitectura.
3. Distribución de paquetes, funcionamiento.
4. Capturas de pantalla del funcionamiento.
5. Dificultades y Soluciones.
6. Bibliografía / Webgrafía, Links.

Marco teórico.

El presente sistema consiste en una aplicación de consola, desarrollada en Java, destinada a la gestión de un catálogo de productos estructurado por categorías, el control de usuarios con roles definidos y la administración de pedidos de compra con sus respectivos detalles.

A continuación, se describen las principales tecnologías y conceptos aplicados:

- Java y Programación Orientada a Objetos (POO): Se utilizan conceptos clave como la herencia (a través de la clase abstracta Base), el polimorfismo y la abstracción mediante la interfaz Calculable, la encapsulación estricta de atributos y la modularización en paquetes bien definidos.
- Bases de Datos Relacionales y MySQL: La información reside de manera persistente en un motor MySQL estructurado en tablas normalizadas vinculadas mediante restricciones de clave primaria y foránea (relaciones de uno a muchos entre categorías-productos, usuarios-pedidos y pedidos-detalles).
- JDBC (Java Database Connectivity) Nativo: Se prescinde de ORMs complejos en favor de una conexión directa de bajo nivel con el driver `com.mysql.cj.jdbc.Driver`. Esto permite comprender a fondo el ciclo de vida de los objetos `Connection`, `PreparedStatement` y `ResultSet`.
- Patrón DAO (Data Access Object): Centraliza la lógica de persistencia. Al implementar una interfaz parametrizada (`GenericDAO<T>`), se abstrae por completo el comportamiento CRUD (Create, Read, Update, Delete) de la lógica de negocio.

Decisiones Técnicas y Arquitectura:

Para este proyecto elegimos una arquitectura en capas desacopladas. Esta separación de responsabilidades simplifica el mantenimiento a largo plazo, mejora la escalabilidad y permite probar cada componente de manera independiente.

Distribución de paquetes:

Paquete	Propósito	Clases Relevantes
config	Manejo centralizado y estático de la conectividad con el motor de base de datos.	DatabaseConnection
entities	Modelado del dominio del problema. Contiene las clases de negocio y contratos de comportamiento.	Base, Calculable, Categoria, Producto, Usuario, DetallePedido, Pedido

dao	Capa de persistencia pura encargada de las consultas SQL, mapeo de tuplas a objetos y control de transacciones.	GenericDAO, CategoriaDAO, ProductoDAO, UsuarioDAO, PedidoDAO, DetallePedidoDAO
services	Capa intermedia que valida reglas de negocio antes de impactar los datos.	CategoriaService, DetallePedidoService, PedidoService, ProductoService, UsuarioService
ui	Interfaz de usuario por consola e hilado principal de dependencias.	CategoriaMenu, Menu, PedidoMenu, ProductoMenu, UsuarioMenu
enums	Tipos estáticos que limitan y validan los valores permitidos para ciertos atributos de las entidades.	Rol, Estado, FormaPago
exception	Centralización de excepciones personalizadas para capturar errores de bajo nivel y propagarlos de forma limpia.	DAOException, ValidationException

Capturas de pantalla del funcionamiento:

Command Prompt - E:\Programacion\maven-3.9.16\bin\mvn.cmd exec:java

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 1

--- CATEGORÍAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2

-- Crear categoría --
Nombre: Electronicos
Descripción: electricos chuchis
? Categoría creada.

```

```

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1

ID  Nombre          Precio  Stock  Categoría
-----
Producto [nombre=Electronicos, precio=999.99, descripcion=muy lindos, stock=2, imagen=foto.png, disponible=true, categoria=Electronica]

--- PRODUCTOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 3

-- Editar producto --
ID de producto: 1
Actual: Producto [nombre=Electronicos, precio=999.99, descripcion=muy lindos, stock=2, imagen=foto.png, disponible=true, categoria=Electronica]
Categorías disponibles:
  Categoría [nombre=Electronicos, descripcion=electricos chuchis, listaProductos=null]
Nuevo nombre (Enter = mantener): Samsung S24
Nuevo precio (Enter = mantener):
Nueva descripción (Enter = mantener):
Nuevo stock (Enter = mantener):
Nueva imagen (Enter = mantener):
¿Disponible? S/N (Enter = mantener): s
Nuevo ID categoría (Enter = mantener):
? Producto actualizado.

```

```

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 4

--- PEDIDOS ---
1. Listar
2. Crear pedido
3. Ver detalle de pedido
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: 2

-- Crear pedido --
Usuarios disponibles:
ID de usuario: 1
Forma de pago:
  1. TARJETA
  2. TRANSFERENCIA
  3. EFECTIVO
Seleccione: 2
Error: No existe un usuario activo con id: 1

--- PEDIDOS ---
1. Listar
2. Crear pedido
3. Ver detalle de pedido
4. Actualizar estado / forma de pago
5. Eliminar
0. Volver
Seleccione: _

```

Dificultades y Soluciones:

A medida que fuimos desarrollando y uniendo los componentes del sistema, nos topamos con tres grandes obstáculos que requirieron un ajuste en la lógica:

- La atomicidad de los pedidos: El principal problema era que un pedido se compone de varias filas en la base de datos (el encabezado y sus detalles). Si la cabecera se guardaba bien pero un detalle fallaba, la base de datos quedaba inconsistente. Lo resolvimos manejando la transacción de forma manual en el *PedidoDAO*. Usando *conn.setAutoCommit(false)*, agrupamos todas las inserciones bajo una misma conexión compartida. Si todo sale bien se confirma la operación, y si salta alguna excepción, el *rollback()* cancela los cambios para no dejar datos huérfanos.
- El problema del *StackOverflowError*: Al tener relaciones en ambos sentidos (la categoría tiene productos y el producto pertenece a la categoría), el método *toString()* de Java entraba en un bucle recursivo sin fin al intentar imprimir los objetos. Para romper este ciclo, modificamos la clase *Producto* para que su método de visualización no renderice la categoría entera, sino únicamente su nombre a través de *categoria.getNombre()*.
- Historiales rotos por eliminaciones físicas: No podíamos usar un *DELETE* tradicional para borrar usuarios o categorías porque romperíamos las claves foráneas de pedidos anteriores. La solución fue adoptar el patrón de borrado lógico. Gracias a la herencia de la clase *Base*, ahora usamos sentencias *UPDATE* para marcar el atributo *eliminado* como verdadero. De esta manera, el registro sigue existiendo para proteger los datos históricos, pero queda oculto en el día a día del sistema mediante filtros *WHERE eliminado = false*.

Bibliografía / Webgrafía:

- Conexión Java + MySQL con JDBC:
https://www.youtube.com/watch?v=BzVkX_67BFQ
- MySQL Enterprise. (2026). MySQL Connector/J Developer Guide. Manual técnico del driver oficial JDBC de MySQL.
- Oracle Corporation. (2026). Java JDBC API Specification and Guide. Documentación oficial de conectividad de bases de datos.

Links:

- Repositorio: <https://github.com/amorillas19/utn-programacion2-integrador>